



INF1009
OOP Group Project Part 2
Group 2: Cyber Scouts

Student Name	Student ID	Email	Signature
Cody Tan De Xuan	2503322	2503322@sit.singaporetech.edu.sg	
Muhammad Hasif Bin Mohd Faisal	2500619	2500619@sit.singaporetech.edu.sg	
Tan Jian Ying Ivan	2501354	2501354@sit.singaporetech.edu.sg	
Natalie Peh	2501526	2501526@sit.singaporetech.edu.sg	
Alvin To Yie Kwan	2502772	2502772@sit.singaporetech.edu.sg	
Lee Hong Yih	2501110	2501110@sit.singaporetech.edu.sg	

Declaration:

We hereby pledge that this Group Project is not plagiarized and has been written wholly as a result of our own research and compilation of information

Date of Submission: 27/03/2026

Table of Contents

1. Introduction

1.1 Engine Overview and Improvements Since Part 1

1.2 Game Overview

2. System Design and Implementation

2.1 Overall Design

2.1.1 Layered Architecture

2.1.2 Engine Layer & Updated Engine Layer UML

2.1.3 Game Layer & Updated Game Layer UML

2.1.4 Game Files and Classes

2.1.5 User/Scene flow

2.1.6 Design Patterns

Factory Method: EntityFactory

Strategy Pattern: InputHandler, SceneRenderer, ResourceLoader

Observer Pattern: ConfigListener/ConfigDispatcher

Template Method: Scene lifecycle

Singleton: ConfigManager

Adapter / Component (ECS component model via ComponentAdapter)

2.1.7 SOLID Principles

2.2 Key Feature

2.3 Innovation

[Practical Value and Measurable Impact](#)

3. Limitations and Future Improvements

3.1 Technical Limitations

3.2 Functional and Design Limitations

Limited Configuration and Accessibility

Presentation Constraints and Content Scope

3.3 Technical Improvements

3.4 Functional and User-Facing Improvements

Accessibility and Presentation Enhancements

Gameplay Expansion

4. Team Contributions

Appendix A - Figures

References

1. Introduction

1.1 Engine Overview and Improvements Since Part 1

Cyber Scouts engine is a custom, reusable 2D game engine that runs on the libGDX platform. Critically, the engine's architectural core, including scene management, the entity-component system, configuration, and dependency management, has no direct dependency on libGDX. libGDX is isolated to the IO boundary layer only, meaning the engine's design logic could be ported to a different platform by replacing only the input and output adapters.

It is structured into two strict layers: an Engine layer that provides generic, game-agnostic services, and a Game layer that depends on the engine but never vice versa. The engine manages the scene transitions, input/output, entity components, collision detection, audio, and configuration all through well-defined abstractions.

Since part 1, the engine has undergone significant improvements across five areas:

Encapsulation Refactor. In Part 1, the engine's example scenes accessed the engine state through direct field references. In Part 2, all engine state is accessed exclusively through **getContext()** and typed accessor methods (e.g., **getContext().getInputManager().getKeyboard()**). This enforces the principle that subclasses interact only through defined interfaces, not through internal state.

Scene Strategy Delegation. The **Scene** abstract class was extended with three abstract handler slots such as **InputHandler**, **SceneRenderer**, and **ResourceLoader**. Each scene now sets its own concrete handlers in its constructor, separating input logic, rendering logic, and asset management into independent, swappable strategies. This was not present in Part 1.

Configuration System. A **ConfigManager** was introduced, backed by **IConfigStore**, **IConfigLoader**, and **IConfigDispatcher** interfaces. Settings are loaded from **engine.properties** at startup and broadcast to registered **ConfigListener** Observers when changed, enabling live volume and display updates from the settings scene without any scene-to-scene coupling.

Audio Support. The **OutputManager** now delegates to an **Audio** class supporting **playMusic()**, **playSound()**, and **stopMusic()**. Background music and sound effects are loaded and unloaded per-scene via the **ResourceLoader** lifecycle.

Mouse Coordinate Transformation. The **InputManager** now accepts a coordinate transformer lambda (**screenToWorld**) so mouse hit positions are always mapped correctly to world coordinates, regardless of viewport scaling or window resize.

Expanded Scene Ecosystem. Part 1 had a basic menu and background scene. In Part 2, there is an addition of **CharacterSelectScene**, **HowToPlayScene**, **PauseScene**, **SettingsScene**, **GameOverScene**, **LeaderboardScene**, and **StartGamePromptScene**, which are all integrated into the stack-based **SceneManager** classes, which are all integrated into the stack-based **SceneManager**.

Storage Abstraction Layer. A **StorageProvider** interface was added to the engine's IO layer, backed by **StorageProviders.preferences()**. Both **LeaderboardManager** and the new **RunSaveManager** use this abstraction rather than calling libGDX Preferences directly, making persistence testable and platform-swappable.

1.2 Game Overview

Cyber Scouts is a fast-paced dropper game with an embedded cybersecurity education layer. The player controls a character at the bottom of the screen, moving left and right to catch safe cyber items (good bytes, safe emails) while avoiding threats (phishing hooks, ransomware, malware). Catching a **frenzy_orb** activates a fast-paced Frenzy phase with harder enemies. Catching certain special entities

pauses the game and presents a multiple-choice cybersecurity question. Correct answers reward the player; incorrect answers will cost a life. Every 200 points, the game pauses to offer the player a choice of three power-up cards.

Problem addressed: General public awareness of cybersecurity threats is low. Most people cannot identify phishing emails, understand what ransomware is, or know what a VPN does. Cyber Scouts addresses this by embedding real cybersecurity MCQ questions directly into a gameplay loop, making learning active rather than passive.

Target audience: Ages 13 and above, particularly students and young adults who may engage with games but have limited exposure to cybersecurity concepts. The arcade-style gameplay lowers the barrier to entry while the quiz mechanic delivers genuine educational content.

Scope and goals:

- Build a reusable engine layer (scene management, ECS, IO, config) that could support future games
- Implement a complete, playable cybersecurity arcade game with multiple characters, progressive difficulty, frenzy mode, quiz overlay, and a persistent leaderboard
- Demonstrate all OOP principles and design patterns learned in INF1009 through concrete, documented implementation choices.

2. System Design and Implementation

2.1 Overall Design

2.1.1 Layered Architecture

The system is divided into two layers with a strict one-way dependency: the engine layer and the game layer.

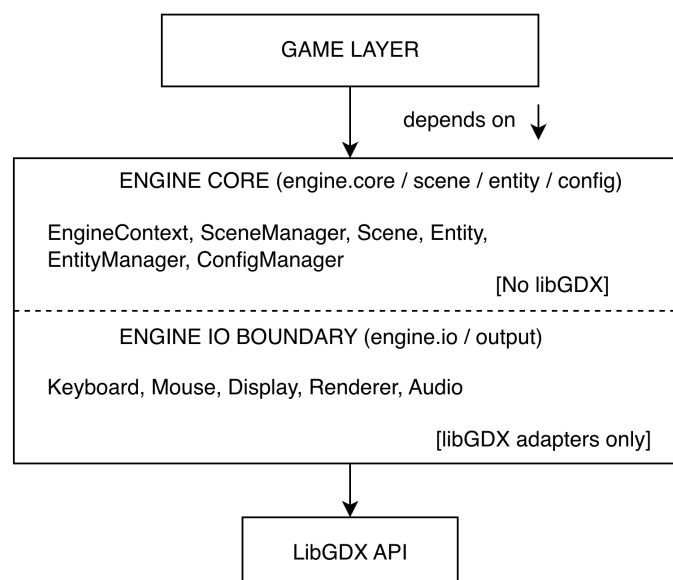


Figure 1. Architectural Layer

The engine layer has zero knowledge of **Cyber Scouts'** game concepts. This separation makes the engine reusable for a different game with no changes.

2.1.2 Engine Layer & Updated Engine Layer UML

The engine layer contains only general, reusable components with no game-specific code, and can function independently of any concrete game implementation. `EngineContext` acts as the central coordinator, creating and wiring all engine components together through composition. The `engine.scene` component provides `Scene` as the abstract base class, `SceneManager` for stack-based scene transitions, and three strategy interfaces — `InputHandler`, `SceneRenderer`, and `ResourceLoader`. The `engine.entity` component defines the ECS, where `Entity` owns its components through composition and `EntityManager` tracks entities via aggregation, with `ComponentAdapter` as the marker interface all components implement.

The `engine.io` component forms the IO boundary — the only part of the engine that interacts with `libGDX` directly. `EngineColor` and `StorageProvider` act as abstraction barriers, preventing `libGDX` types from leaking above this boundary. The `engine.config` component separates configuration responsibilities across three interfaces and notifies `OutputManager` of changes through observer classes. Finally, `engine.collision` and `engine.movement` are self-contained components providing collision detection and velocity-driven movement with no dependency on the game layer.

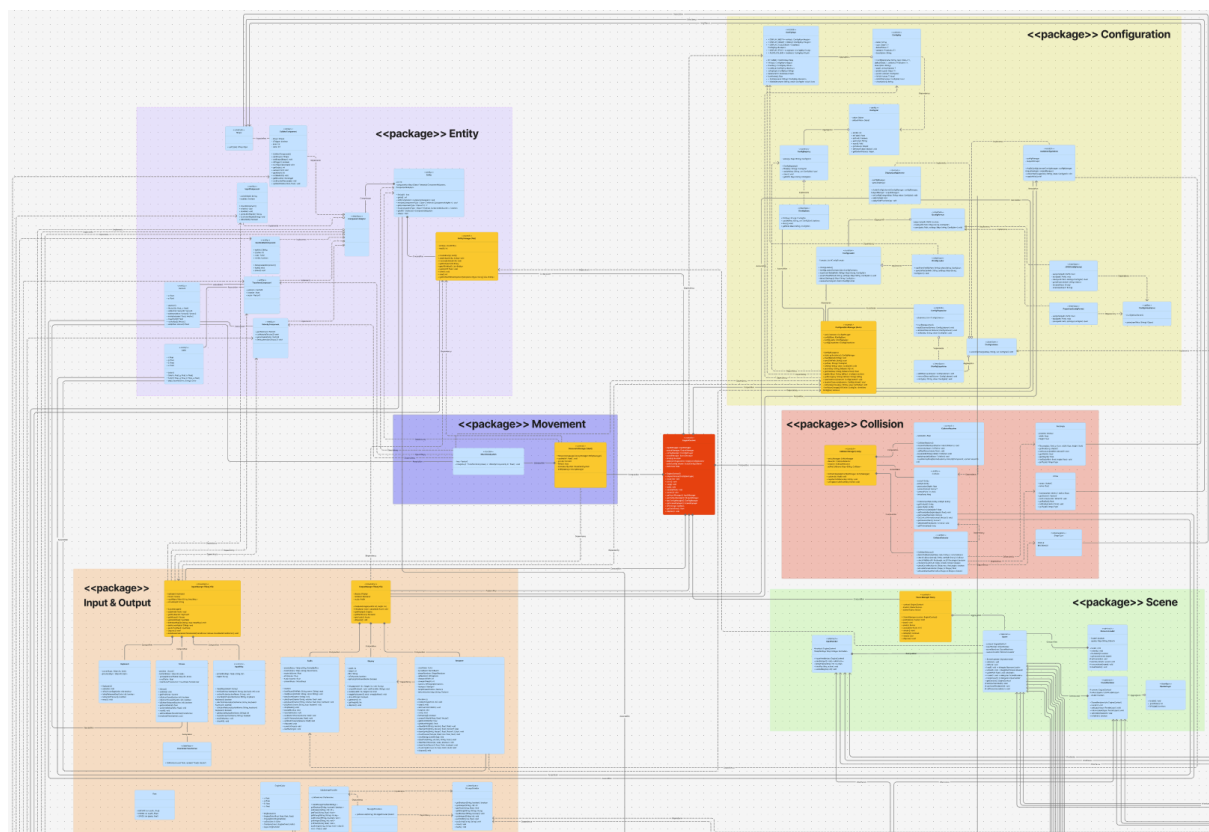


Figure 2. Game Engine UML

2.1.3 Game Layer & Updated Game Layer UML

The game layer sits above the engine layer and keeps game logic independent from direct `libGDX` calls. Classes in `game.scenes` extend `Scene`, with `GamePlayScene` coordinating core systems (`EntityFactory`, `QuizManager`, `LeaderboardManager`) via composition. `EntityType`, `CharacterType`, and `BuffType` define reusable domain enums for factories and scenes, while `FallingComponent`,

GameEntityComponent, and HealthComponent extend ECS behaviour through ComponentAdapter without changing engine internals. QuizManager composes QuizBank, LeaderboardManager persists scores through StorageProvider, and RunSaveManager captures full run snapshots for resume support. GameUiTheme standardizes fonts and colors across scene renderers for consistent UI presentation.

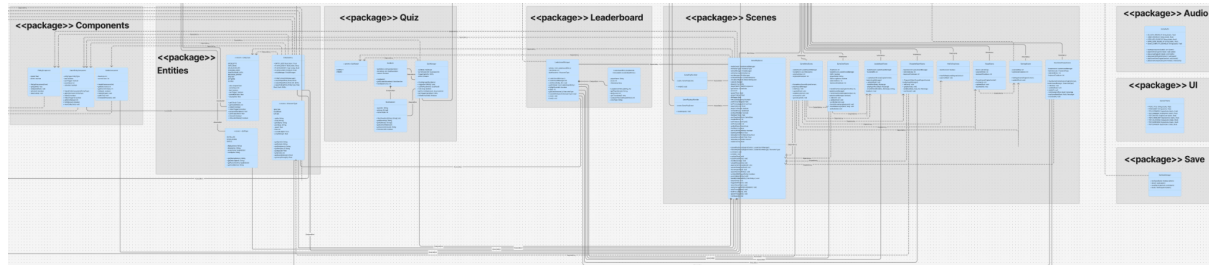


Figure 3. Game Layer UML

Full UML Link:

<https://www.figma.com/board/iJCQOmu0oKesB6CrCTPMoD/Abstract-Engine-UML-Class-Diagram?node-id=0-1&t=Tq9RBgT49dcDWzsj-1>

2.1.4 Game Files and Classes

The main game files and classes are:

game.audio

File	Classes	Purpose
GameAudio.java	GameAudio	Central registry of all game sound IDs and file paths; provides static helper methods so any scene can trigger audio without hardcoding paths.

game.components

File	Classes	Purpose
FallingComponent.java	FallingComponent	ECS component storing fall speed and active state for dropping entities.
GameEntityComponent.java	GameEntityComponent	ECS component tagging an entity with its type, bad/good flag, quiz-trigger flag, score value, and collected state.
HealthComponent.java	HealthComponent	ECS component tracking the player's current and maximum lives, with methods to take damage, gain a life, or raise the life cap.

game.entities

File	Classes	Purpose
------	---------	---------

BuffType.java	BuffType (enum)	Enum of the three buff cards offered every 200 points (Extra Life, Speed Surge, Shield), each with a display name, description, colour, and sprite.
CharacterType.java	CharacterType (enum)	Enum of the three selectable player characters (Specter, Guardian, Cipher), each with its own speed, lives, score multiplier, and jump strength.
EntityFactory.java	EntityFactory	Factory class that assembles fully-configured Entity objects by attaching the correct ECS components, keeping scene code free of wiring details.
EntityType.java	EntityType (enum)	Enum of every falling entity type (good, bad, frenzy-only, special trigger), each carrying its colour, score value, harm flag, and speed multiplier.

game.leaderboard

File	Classes	Purpose
LeaderboardEntry.java	LeaderboardEntry	Immutable value object holding one leaderboard record (name, score, timestamp); implements Comparable for highest-first sorting.
LeaderboardManager.java	LeaderboardManager	Manages the sorted top-10 leaderboard and persists it to device storage via StorageProvider.

game.quiz

File	Classes	Purpose
QuizBank.java	QuizBank	Pool of cybersecurity-themed MCQ questions; draws questions randomly without immediate repetition.
QuizManager.java	QuizManager	State machine (IDLE / ACTIVE) that controls when a quiz is triggered, which question is shown, and how the result is resolved.
QuizQuestion.java	QuizQuestion	Immutable value object representing one MCQ question: the question text, four options, and the correct answer index.
QuizResult.java	QuizResult (enum)	Enum representing the outcome of a quiz answer (CORRECT / WRONG).

game.save

File	Classes	Purpose
RunSaveManager.java	RunSaveManager, RunSnapshot (inner), FallingEntitySnapshot	Persists and loads the in-progress game runs to device storage. RunSnapshot holds the full game state snapshot; FallingEntitySnapshot captures the state of each active falling entity at save time.

	FallingEntitySnapshot (inner)	
--	-------------------------------	--

game.scenes

File	Classes	Purpose
CharacterSelectScene.java	CharacterSelectScene	Scene for choosing a playable character; supports keyboard and mouse navigation across the three character cards.
GameMenuScene.java	GameMenuScene	Main menu scene with Start Game, Leaderboard, and Exit options.
GameOverScene.java	GameOverScene	Displayed when the player loses all lives; handles name entry for the leaderboard, then navigates to menu or leaderboard.
GamePlayInputHandler.java	GamePlayInputHandler	Handles all player input during active gameplay (movement, jump, quiz answers, pause trigger).
GamePlayRenderer.java	GamePlayRenderer	Draws the full gameplay view: entities, player, HUD, quiz overlay, buff card selection, and frenzy banner.
GamePlayScene.java	GamePlayScene, GameState (inner enum)	Core gameplay scene owning the main game loop. GameState enum defines all possible states: PLAYING, FRENZY, QUIZ, QUIZ_FEEDBACK, BUFF_SELECT, TRANSITION_TO_FRENZY, GAME_OVER, WIN.
HowToPlayScene.java	HowToPlayScene	Two-page tutorial screen explaining controls, entity types, and scoring.
LeaderboardScene.java	LeaderboardScene	Displays the top scores in a ranked table with buttons to return to the menu or play again.
PauseScene.java	PauseScene	Pause overlay pushed on top of GamePlayScene when ESC is pressed; offers Resume, Settings, and Exit to Menu.
SettingsScene.java	SettingsScene	Settings screen with volume sliders and resolution dropdown; persists changes via ConfigManager.
StartGamePromptScene.java	StartGamePromptScene	Shown when a saved run exists; lets the player continue or start a fresh game.

game.ui

File	Classes	Purpose
GameUiTheme.java	GameUiTheme	Shared UI theme constants (font keys and colour palette) for consistent text styling across all game scenes.

2.1.5 User/Scene flow

User Game Flow

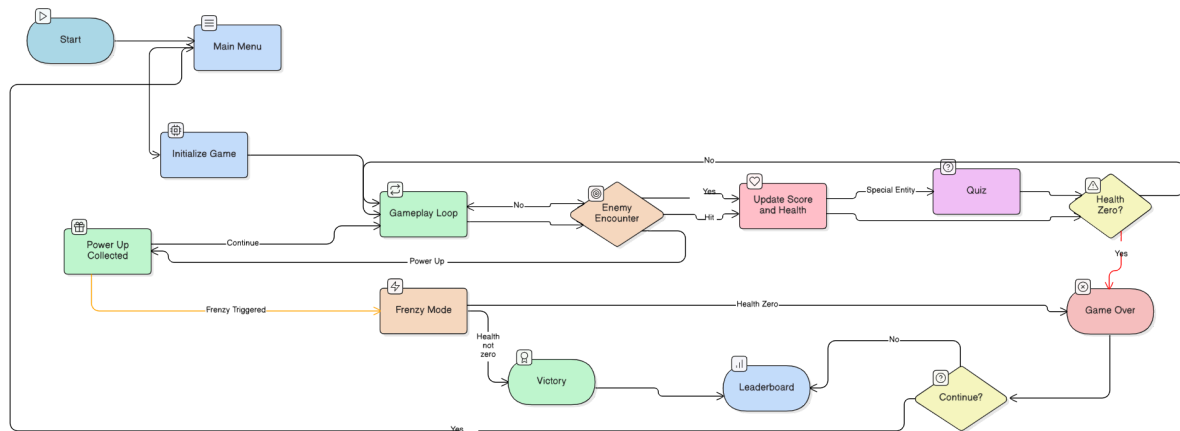


Figure 4. User Game Flow

Upon launching the game, the player would be on the game's main menu, which presents the player with 5 navigational options: Start game, Leaderboard, Tutorial, Settings, and an Exit button.

Leaderboard

It stores and displays scores from previous runs, allowing players to review their performance over time.

Tutorial (How to play)

The tutorial menu provides the player with a guide to the game with information on the game's objective, controls, safe entities, dangerous entities, frenzy mode and character styles.

Settings

The settings saves the player's configuration state for the game which includes: Master Volume, Music Volume, SFX Volume, 5 different Screen Resolutions and a Fullscreen option. The player's changes will be in effect upon exiting the settings menu.

Game Flow

Upon starting the game, the player would be brought to the character selection menu. After selecting the character the player would be led to the main game where it loads the chosen character assets, spawns entities, initialising the default score and health for the chosen character.

The gameplay operates in a continuous cycle with both good and bad entities spawning and dropping, inputs from W, A, D keys are used to move the in-game character. When the character touches an entity, the following will correspond:

- Bad Entity: Triggers "Apply Bad Effect" (e.g., losing health)
- Special Entity (Quiz): Triggers a "Quiz Event" (detailed below)
- Good Entity: Triggers "Apply Good Effect" (e.g., gaining points/health)

Additionally, this game features two unique modes:

1. Quiz System

Amid all the entities, there is a chance that when the character interacts with the entity, the game will enter a quiz system stage where a question picked from a question pool is presented to the player. If the player picks the correct option, the player is given buffs to improve the gameplay, if the incorrect option is picked then the player takes damage. The player will then be returned to the core gameplay loop.

2. Frenzy Mode

This is triggered upon colliding with a Frenzy orb, which activates the game's Frenzy Mode. This mode is a timed mode with faster entity drop speeds, point multipliers and introduces 2 new bad entities, essentially making the gameplay more challenging and rewarding. After the limited duration is up, the game returns to its core gameplay loop once again.

The game concludes when the player's health is depleted or when the number of entity quota collected is met. The player would be prompted afterwards to input a name to record his current run/performance.

2.1.6 Design Patterns

The project applies several design patterns to keep gameplay logic modular, extensible, and maintainable.

1. **Factory Method:** EntityFactory

EntityFactory.createFallingEntity() and createPlayer() encapsulate all entity construction. Callers supply only a type and position; they receive a fully assembled, ready-to-use entity. Adding a new entity type requires only a new EntityType constant and, optionally, a line in EntityFactory.

GamePlayScene only knows type and position:

```
entityFactory.createFallingEntity(EntityType.R00TKIT, x, SPAWN_Y, speed);
```

The game uses the Factory Method pattern to encapsulate entity construction.

EntityFactory.createPlayer() builds a ready-to-use player entity by assembling its required components (transform, health, type), while EntityFactory.createFallingEntity() constructs falling entities from an EntityType and parameters such as spawn position and fall speed. Callers request "what they need" (player or typed falling entity) without needing to know the component assembly steps.

2. **State Pattern:** GameState enum and QuizManager

GamePlayScene defines an inner enum GameState with 8 states:

```
PLAYING → TRANSITION_TO_FRENZY → FRENZY → PLAYING
PLAYING/FRENZY → QUIZ → QUIZ_FEEDBACK → (previous state or GAME_OVER)
PLAYING/FRENZY → BUFF_SELECT → (previous state)
PLAYING → WIN
PLAYING/FRENZY → GAME_OVER
```

Both update(dt) and handleInput() switch on gameState, making every valid and invalid state transition explicit in the code. QuizManager independently maintains its own IDLE/ACTIVE state with an active boolean.

Separately, QuizManager maintains its own state machine: a quiz is either IDLE or ACTIVE, represented by the active boolean. triggerQuiz() is a guarded transition into ACTIVE (does nothing if already active), and submitAnswer() transitions back to IDLE after producing a QuizResult.

3. **Strategy Pattern:** InputHandler, SceneRenderer, ResourceLoader

All three abstract classes. **GamePlayScene** provides **GamePlayInputHandler** (Keyboard movement) + jump + quiz answer selection by keyboard or mouse), **GamePlayRenderer** (entities, HUD, quiz overlay, buff card overlay, feedback banners), and an anonymous **ResourceLoader** (audio load/unload), **Scene.render()** calls **sceneRenderer.render()** without knowing which renderer is active.

The engine's Scene class composes three interchangeable strategy objects: InputHandler, SceneRenderer, and ResourceLoader. Each scene selects implementations appropriate to its role without modifying engine code. For example, GamePlayScene installs a gameplay-specific input strategy (GamePlayInputHandler) and rendering strategy (GamePlayRenderer), and provides a ResourceLoader that controls scene-level resource lifecycles (e.g., loading/unloading audio). The engine stays decoupled: Scene.render() simply calls sceneRenderer.render() and does not need to know which concrete renderer is active.

4. **Observer Pattern:** ConfigListener/ConfigDispatcher

DisplayConfigListener and **AudioConfigListener** implement **ConfigListener** and register with **ConfigManager** during EngineContext construction. When the settings scene changes a config value, **ConfigDispatcher.notifyObservers()** calls each listener's **onConfigChanged()**. **AudioConfigListener** immediately applies the new volume to the **OutputManager**. The settings scene holds no direct reference to **OutputManager**.

5. **Template Method:** Scene lifecycle

The Scene abstraction defines a fixed lifecycle shape with overridable hooks (onEnter(), onExit()) and an abstract update(float dt) that all concrete scenes must implement. SceneManager enforces lifecycle ordering during push/pop operations, while scene subclasses customize only the relevant hook methods and update behavior.

SceneManager always calls **load()** → **onEnter()** when pushing, and **onExit()** → **unload()** when popping. These hooks are defined on the **Scene** with empty default implementations. Subclasses override only what they need. **update(float dt)** is declared **abstract**, forcing all subclasses to provide their own game logic.

6. **Singleton:** ConfigManager

ConfigManager.getInstance() implements a singleton to ensure a single configuration source across the application lifetime. EngineContext obtains the instance and uses it as the shared configuration dependency, preserving global uniqueness while still allowing tests to inject an alternative ConfigManager constructor when needed.

7. **Adapter / Component (ECS component model via ComponentAdapter)**

The engine implements ECS-style composition using a component marker interface, ComponentAdapter. Entities are built by attaching multiple components (e.g., transform, velocity, collider, health), allowing systems to operate on shared component contracts rather than hard-coded entity subclasses. This is an Adapter/Component-style composition approach: features are added by “adapting” an entity's capabilities through attached components, enabling flexible combinations (e.g., a falling entity + gameplay tag + collision bounds) without inheritance-heavy design.

2.1.7 SOLID Principles

The architecture reflects SOLID principles through its separation of concerns, interface-driven design, and compositional structure.

Single Responsibility Principle

The project demonstrates the Single Responsibility Principle by assigning each class or component a focused and well-defined responsibility. For example, `EntityFactory` handles entity assembly, `LeaderboardManager` manages score persistence/ranking, `QuizManager` handles quiz flow and answer resolution, and `HealthComponent` tracks life state. A notable improvement was separating gameplay input and rendering into `GamePlayInputHandler` and `GamePlayRenderer`, reducing responsibility concentration in `GamePlayScene` and improving maintainability.

A clear example of this principle can be seen in the refinement of the gameplay implementation. Earlier in development, too many concerns were concentrated within `GamePlayScene`, including gameplay coordination, input handling, and rendering. This was later improved by extracting `GamePlayInputHandler` and `GamePlayRenderer` into separate classes. As a result, input-related changes are now isolated from rendering-related changes, while `GamePlayScene` remains focused on coordinating gameplay state and progression. This restructuring improved maintainability and reduced the complexity of modifying or debugging gameplay code.

Open/Closed Principle

The project reflects the Open/Closed Principle by allowing several parts of the architecture to be extended without requiring modification to their core management classes. One example is the scene system. New scenes can be introduced by extending the abstract `Scene` class and registering or pushing them through `SceneManager`, without changing the internal design of `SceneManager` itself. Similarly, the configuration subsystem supports extension through centralised keys and abstractions, allowing new configuration values to be added without changing the overall structure of `ConfigManager`.

The project also supports extension in gameplay-related areas through controlled abstractions such as `EntityType` and `BuffType`. These structures make it easier to introduce new entity or buff types in an organised and predictable way. However, this should not be overstated, as some gameplay extensions still require updates to factory mappings or handling logic. As such, the design does not represent a perfect zero-modification example of the Open/Closed Principle. Nevertheless, it still promotes extensibility by encouraging changes to be introduced through centralised abstractions instead of scattered hardcoded logic.

Liskov Substitution Principle

The project follows the Liskov Substitution Principle through its use of abstract scene and component contracts. All game scenes extend the abstract `Scene` base class and are managed uniformly by `SceneManager`. This means that scenes such as `GameMenuScene`, `CharacterSelectScene`, `GamePlayScene`, `PauseScene`, `SettingsScene`, `HowToPlayScene`, `LeaderboardScene`, `StartGamePromptScene`, and `GameOverScene` can all be treated as `Scene` objects without requiring special-case logic. `SceneManager` can invoke `handleInput()` and `update(dt)` on the active scene regardless of its concrete subclass, while rendering is handled separately through the scene lifecycle.

A similar pattern appears in the entity-component design. Components implementing `ComponentAdapter` can be stored in and retrieved from an `Entity` through the same generic API. This allows different component implementations to be used interchangeably without changing the surrounding logic. As a result, valid subclasses and implementations can replace one another without breaking expected behaviour, which is the core idea behind Liskov substitution.

Interface Segregation Principle

The project demonstrates the Interface Segregation Principle by favouring small, focused interfaces over large, general-purpose ones. This is particularly visible in the configuration subsystem. Instead of combining multiple responsibilities into a single broad interface, configuration behaviour is separated into `IConfigStore`, `IConfigLoader`, and `IConfigDispatcher`. Each interface represents one specific concern, allowing classes to depend only on the behaviour they actually require.

This principle is also evident in supporting abstractions such as `ConfigListener`, which contains only a single method, `onConfigChanged()`, for classes that need to respond to configuration updates. Likewise, `ComponentAdapter` acts as a lightweight ECS contract without forcing components to implement unrelated behaviour. This design reduces unnecessary dependencies, keeps the system modular, and helps prevent classes from being burdened with methods they do not need.

Dependency Inversion Principle

The project applies the Dependency Inversion Principle by ensuring that higher-level systems depend on abstractions rather than concrete implementations. For example, `SceneManager` manages scenes through the abstract `Scene` type instead of depending on specific scene subclasses. Within the scene architecture, the base `Scene` class works with abstract roles such as `InputHandler`, `SceneRenderer`, and `ResourceLoader`, while concrete scenes provide the actual implementations through the protected setter methods defined by the base class. This allows scene-specific behaviour to vary without requiring changes to the engine-level framework.

The configuration subsystem follows the same principle. `ConfigManager` depends on abstractions such as `IConfigStore`, `IConfigLoader`, and `IConfigDispatcher` rather than being tightly coupled to a single concrete implementation. In addition, `EngineContext` is injected into each scene through the constructor, allowing scenes to access engine services through a shared abstraction rather than directly constructing dependencies. The same pattern appears in persistence-related features, where classes such as `LeaderboardManager` and `RunSaveManager` depend on the `StorageProvider` abstraction. Together, these choices improve flexibility, testability, and separation between high-level logic and low-level implementation details.

2.2 Key Feature

Engine Features

CyberScouts is implemented as modular subsystems coordinated by a timer-driven loop. The loop updates spawning, movement, collisions, scoring, and mode transitions. A state model coordinates gameplay and UI transitions across menu, selection, gameplay, pause, settings, game-over, and leaderboard scenes.

1. Major engine-backed features include:
2. Timer-driven gameplay with dynamic pacing
3. Frenzy mode trigger and timed override behavior
4. Type-based entity balancing (speed/size tuning)
5. Horizontal + vertical movement support
6. Quiz integration inside active gameplay
7. Persistent leaderboard and onboarding flow
8. Scene-based UI architecture
9. Extensibility scaffolding for future cards/multiplayer slots
10. Gameplay ends when lives reach zero or the main timer reaches zero.

A dedicated mode system manages gameplay state (active play vs pause/game-over) and special modes such as Frenzy Mode. Frenzy Mode is triggered when the player catches the Frenzy Orb. Once activated, the main timer freezes for 15 seconds, and the game becomes more intense by increasing

the drop/spawn pressure and overall pace. During Frenzy, special threats are introduced, including Rootkit (a frenzy-only threat) and Spyware (a quiz threat that appears during Frenzy).

Falling objects use a typed entity framework for balancing: each entity type is configured with its own gameplay parameters (e.g., movement speed tuning and visual size), enabling distinct difficulty without rewriting the core loop. Player control supports both horizontal movement and vertical/jump mechanics, so the engine supports timing-based repositioning rather than purely left-right play.

Quiz functionality is integrated as event-driven gameplay interrupts. Quiz entities present a question during the round and convert the player's answer into immediate game-state effects:

Correct answers: grant +100 points and +1 life

Wrong answers: grant no additional points/life and deducts a life

Game progression ends when either lives reach 0 (from collisions with bad entities) or the match timer runs out. The leaderboard persists scores using player identity, and a first-time onboarding/tutorial flow is shown automatically. UI is handled through dedicated scenes for settings, pause, character selection, name entry, and game-over. Extensibility scaffolding is included for future card mechanics and multiplayer slots.

2.3 Innovation

Cybersecurity Education Through GamePlay

Cyber Scouts' core innovation is integrating real cybersecurity MCQ questions as a direct consequence of gameplay events, not as a separate mode. When a player catches **ransomware_lock**, they must immediately answer "What is ransomware?" It is thematically tied to the entity they just caught. The quiz cannot be skipped; gameplay is frozen until the player answers. This creates genuine learning moments under mild pressure rather than passive reading. Topics covered (phishing, 2FA, VPNs, firewalls, rootkits, social engineering, zero-day vulnerabilities) map directly to the entity types visible on screen, reinforcing the association.

Three-Layer Escalating Difficulty

Unlike many arcade titles that rely on a single linear difficulty curve, Cyber Scouts uses a compounding multi-layer model that increases challenge more dynamically.

1. Standard Mode Progression
Baseline difficulty scales at fixed intervals (every 15 seconds), gradually increasing entity pressure over time.
2. Frenzy Mode Acceleration
Frenzy introduces a temporary high-intensity phase with a steeper internal ramp, causing faster escalation than standard play.
3. Post-Frenzy Continuity
When Frenzy ends, progression does not reset; the run continues from the accumulated pre-Frenzy difficulty state, preserving momentum and challenge.

In addition, difficulty is partially player-shaped through buff decisions. Persistent upgrades (e.g., stacking Speed Surge) alter control responsiveness and risk handling, meaning each run's difficulty profile is influenced not only by system timers but also by strategic player choices.

Risk/Reward Character Selection

The **CharacterType** enum gives each character a distinct risk/reward profile enforced entirely by data. Cipher's 1.5x score multiplier can produce dramatically higher scores than Guardian's 1.0x, but

with only 2 lives, the margin for error is far smaller. No special casing in game logic is needed; `characterType.getScoreMultiplier()` is called at the point of scoring, and the rest follows. Balance adjustments require only changing the enum constant.

Practical Value and Measurable Impacts

The design supports measurable educational and gameplay outcomes. The system can be evaluated through metrics such as:

1. Quiz accuracy by topic across repeated runs to measure concept retention over time.
2. Wrong-answer frequency by concept category to identify persistent knowledge gaps (e.g., phishing vs. VPN vs. zero-day).
3. Survival and performance trends across sessions to evaluate whether improved understanding translates into better gameplay outcomes.
4. Score distribution by character archetype and buff path to assess strategic diversity, balance, and risk/reward behavior.

Together, these metrics make the innovation operationally valuable: the system is not only engaging to play, but also quantifiable in terms of learning progression and player mastery.

Architectural Innovation and Extensibility

Innovation is reinforced by modular subsystem separation (QuizManager, EntityFactory, RunSaveManager, LeaderboardManager, ECS components). Educational triggers, progression mechanics, and persistence are decoupled, allowing future expansion with minimal architectural disruption. New cybersecurity topics, entities, buffs, or balancing adjustments can be introduced by extending existing data/models and managers rather than rewriting core flow.

3. Limitations and Future Improvements

3.1 Technical Limitations

Incomplete Abstraction and Framework Dependence

The project demonstrates a clear separation between the engine and game layers, but one remaining limitation is that the game layer still uses libGDX classes such as `Vector2` and `Rectangle` directly for positioning, layout, and hitbox calculations. This was a practical development choice, but it means the abstraction boundary between the engine and the framework is not yet fully complete.

Centralised Gameplay and Save Management

Another limitation is that gameplay logic remains relatively centralised. Although gameplay input and rendering responsibilities were separated into supporting classes, `GamePlayScene` still handles multiple major concerns, including gameplay flow, buffs, HUD updates, and run restoration. This was manageable for the current project scope, but in a larger version it could become harder to maintain. Similarly, the save-and-resume system is intentionally lightweight, focusing only on restoring an interrupted run rather than supporting broader persistence features such as multiple save slots or richer save data.

3.2 Functional and Design Limitations

Limited Configuration and Accessibility

The game includes the most important user-facing settings, such as volume adjustment, full-screen support, and resolution selection. However, the configuration system remains focused on core options and does not yet include features such as key rebinding, UI scaling, or additional accessibility controls. This limits flexibility for users with different preferences and needs.

Presentation Constraints and Content Scope

The game preserves a fixed gameplay aspect ratio to maintain a consistent layout and predictable world-space behaviour across different screen sizes. While this improves visual reliability, it can also result in letterboxing on certain displays. In addition, although *CyberScouts* provides a complete and polished gameplay loop with quizzes, frenzy mode, buffs, leaderboard support, multiple characters, and run persistence, the overall content scope remains relatively compact. Replayability could therefore be improved further through greater gameplay variety and progression depth.

3.3 Technical Improvements

Stronger Encapsulation and Modularity

A useful future improvement would be to introduce engine-owned wrapper classes for mathematical structures such as vectors and rectangles. This would reduce direct dependency on framework-specific types and strengthen the separation between the engine and libGDX. Another worthwhile step would be to further modularise gameplay logic by moving concerns such as frenzy handling, buff processing, quiz transitions, and run restoration into smaller dedicated classes, making the codebase easier to maintain, test, and extend.

Expanded Persistence and Testing

The persistence system could also be developed into a richer save model with support for multiple save slots, clearer save information, and more flexible restore behaviour. Automated testing could likewise be expanded, particularly for gameplay-specific scenarios such as frenzy transitions, save restoration, and health-state handling, to improve long-term reliability.

3.4 Functional and User-Facing Improvements

Accessibility and Presentation Enhancements

From the user's perspective, future work could expand the settings system with greater accessibility and personalisation, including input rebinding, scalable interface elements, higher-contrast options, and additional display controls. The visual and audio presentation could also be refined through smoother music transitions, stronger feedback effects, and further polish during key gameplay moments, improving both clarity and immersion.

Gameplay Expansion

The gameplay itself could be extended with a wider range of quiz content, more varied events, stronger character differentiation, and additional entity behaviours. As the current foundation is already stable and well-structured, these would be realistic extensions rather than major redesigns.

4. Team Contributions

Team Member	Major Contributions
-------------	---------------------

Cody	Contributed to the early game concept direction and gameplay ideation, helping shape the structure of the project before later refinements were introduced. This included early prototyping work, design-pattern-oriented structuring in the initial game logic, educational content direction, and support for gameplay framing and progression ideas. Also contributed through asset creation, user testing, and assistance with report-related support, helping validate whether the game flow and user experience were understandable and workable from a player perspective.
Hasif	Contributed mainly to integration, refinement, and submission preparation across both the engine and game layers. This included validating merged code across branches, carrying out UI and UX adjustments, refining code structure, improving encapsulation and layering, resolving gameplay and scene-flow issues, and supporting the final rounds of bug fixing before submission. Also contributed to UML design alignment and consistency checking, helping ensure that the implemented system remained coherent with the intended engine structure and overall project requirements.
Ivan	Contributed to refactoring and improving the engine-side scene and runtime structure, with particular involvement in scene-related adjustments, layout improvements, supporting engine-side organization, and helping refine how the abstract engine components were presented and exercised. Took the lead on report layout and the overall presentation of the written submission, helped organise the report into a clearer and more readable final structure. This contribution supported both the technical side of the engine's scene framework and the clarity of the final documentation.
Natalie	Contributed to both technical and presentation-facing areas of the project. On the implementation side, supported improvements to robustness and user experience, including engine-related enhancement work and refinement of interaction behavior. On the final delivery side, taking charge of preparing the presentation slides, planning the structure of the final presentation, supporting video editing, and contributing visual assets where needed. This helped ensure that the final project was not only functional, but also communicated clearly in presentation format.
Alvin	Executed gameplay ideation, visual asset production, engine refinement and supporting figure creation for the project. This included preparing and refining game art assets used across the interface and gameplay, as well as supporting the generation of diagrams or visual materials needed for explanation and presentation. In addition to visual support, also strengthened the project with technical refinement and adjustment work during the integration phase, helping polish the overall completeness of the project deliverables. Additionally, designed the narrative flow and presentation of the game demo.
Hong Yih	Took primary ownership of extending the abstract engine into the playable game layer and implementing the main gameplay features. This included the initial development of the CyberScouts game structure, core scene flow, interactive gameplay systems, and major game-side mechanics such as character selection, quiz interactions, frenzy-related gameplay, card or buff progression, and general game feature integration. Forming the main functional base of the final game, which was later refined and stabilized through subsequent team integration and review. Contributed to the report and slides and tidying up of the report. Checking code and report and ensuring alignment in all aspects of the report and slides. Innovated and implemented the idea of card upgrades to manipulate current game structure to ensure that every game run is customisable and different from the last.
AITeammate	Clarification of coding concepts, implementation guidance, and debugging assistance

Appendix A - Figures

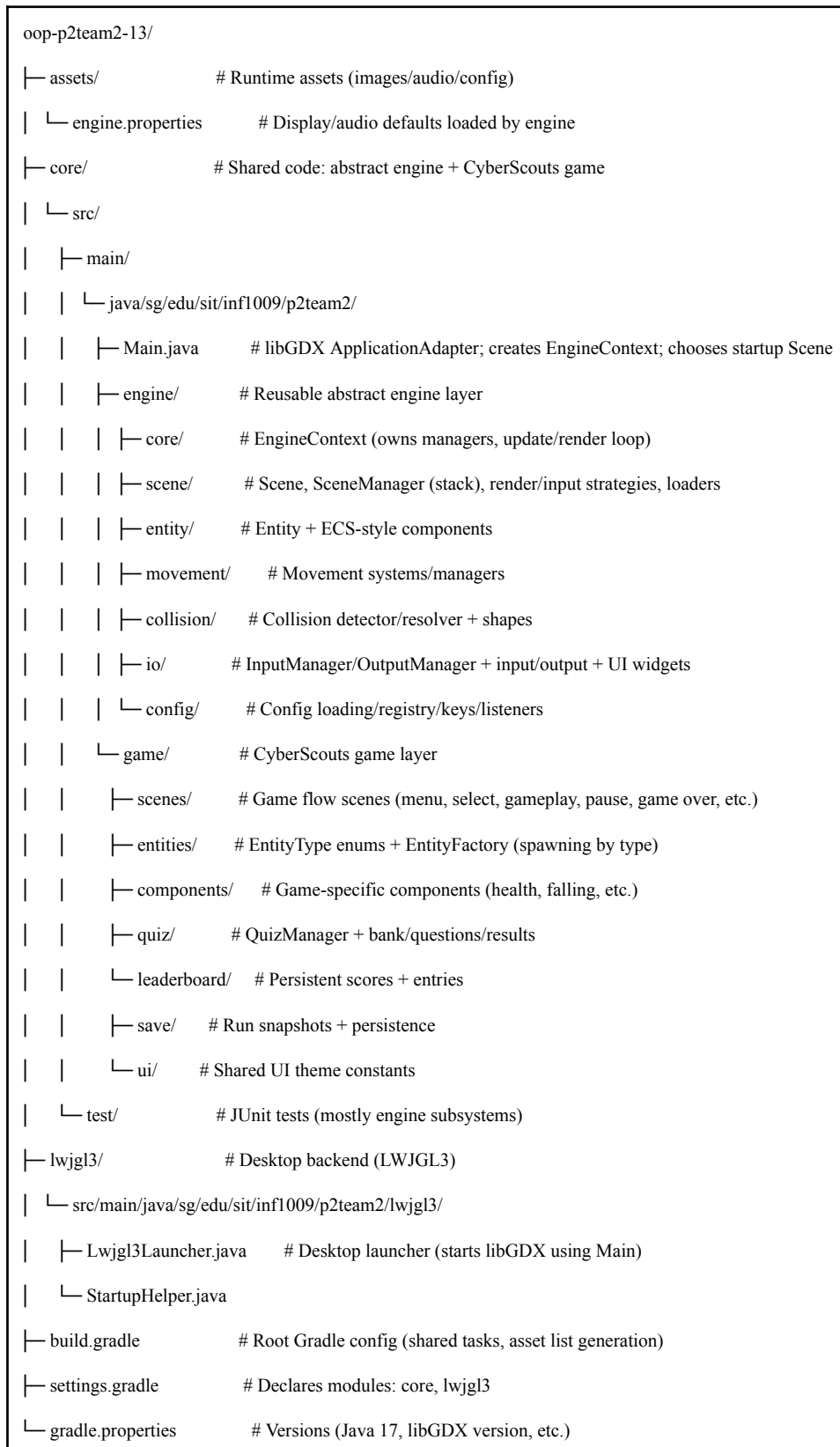


Figure 5. Project File Structure

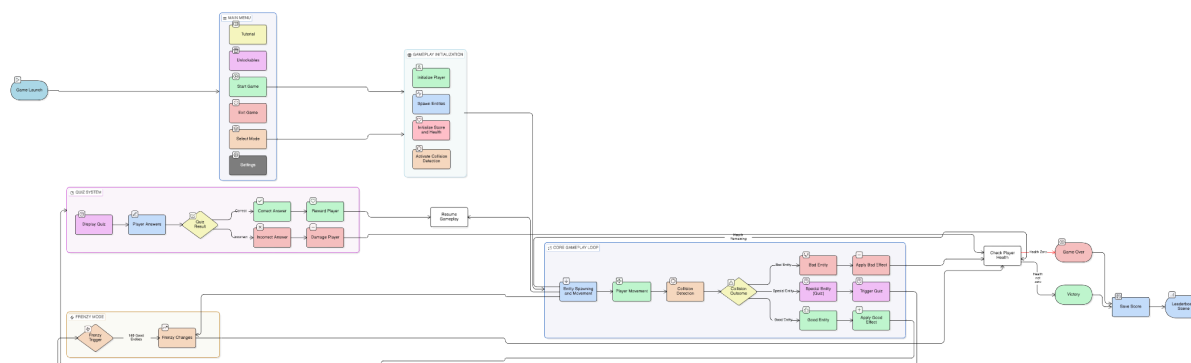


Figure 6. Full Game Flow

References

- Alexandr Zhelanov. (2018, January 11). Law in the city [Audio file]. OpenGameArt. <https://opengameart.org/content/law-in-the-city>
- CJspellsfish. (2024, March 15). Basic retro jump [Audio file]. Freesound. <https://freesound.org/people/CJspellsfish/sounds/727650/>
- colorsCrimsonTears. (2021, April 5). Fanfare - Rpg [Audio file]. Freesound. <https://freesound.org/people/colorsCrimsonTears/sounds/566203/>
- colorsCrimsonTears. (2021, September 5). Gameover [Audio file]. Freesound. <https://freesound.org/people/colorsCrimsonTears/sounds/585797/>
- Dijkstra, E. W. (1986). Science fiction and science reality in computing (EWD952). University of Texas at Austin. <https://www.cs.utexas.edu/~EWD/transcriptions/EWD09xx/EWD952.html>
- Fowler, M. (2006). Separated presentation. Martin Fowler. <https://martinfowler.com/eaaDev/SeparatedPresentation.html>
- Fowler, M. (2013). Local DTO. Martin Fowler. <https://martinfowler.com/bliki/LocalDTO.html>
- GabrielAraujo. (2014, July 13). Failure/wrong action.wav [Audio file]. Freesound. <https://freesound.org/people/GabrielAraujo/sounds/242503/>
- Google. (n.d.). Orbitron. Google Fonts. <https://fonts.google.com/specimen/Orbitron>
- Google. (n.d.). Rajdhani. Google Fonts. <https://fonts.google.com/specimen/Rajdhani>
- Gradle, Inc. (n.d.). Gradle Wrapper. Gradle documentation. https://docs.gradle.org/current/userguide/gradle_wrapper.html
- JUnit Team. (n.d.). JUnit 5 user guide. JUnit. <https://docs.junit.org/5.11.1/user-guide/index.html>
- libGDX. (n.d.). libGDX. <https://libgdx.com/>
- libGDX. (n.d.). Vectors, matrices, quaternions. libGDX Wiki. <https://libgdx.com/wiki/math-utils/vectors-matrices-quaternions>

Oracle. (n.d.). Java Platform, Standard Edition 17 documentation.
<https://docs.oracle.com/en/java/javase/17/>

Ranner. (2019, October 9). UI click [Audio file]. Freesound.
<https://freesound.org/people/Ranner/sounds/487588/>

tricksntraps. (2019, August 19). T & T free cyberpunk pack [Audio file]. OpenGameArt.
<https://opengameart.org/content/t-t-free-cyberpunk-pack>

P2Team2. (2026). *Abstract engine UML class diagram [Diagram]*.
<https://www.figma.com/board/iJCOOmu0oKesB6CrCTPMoD/Abstract-Engine-UML-Class-Diagram?node-id=0-1&t=Tq9RBgT49dcDWzsj-1>

P2Team2. (2026). *Full game & user game flow diagrams*. Eraser.io.
<https://app.eraser.io/workspace/a0N3uueL4xixTHWpdLTg>